```
        "custom_html_strip_filter_analyzer":{
          "tokenizer":"keyword",
          "char_filter":["my_html_strip_filter"]
        }
      },
      "char_filter": {
        "my_html_strip_filter":{
          "type":"html_strip",
          "escaped_tags":["h1"]
        }
      }
    }
  }
}
```

Declares a custom character filter

The escaped_tags attribute ignores parsing <h1> tags in the input field.

This index has a custom analyzer consisting of an `html_strip` character filter. The `html_strip` is extended in this example to use the `escaped_tags` option, which specifies that `<h1>` tags should be left untouched. To test this, we can run the following code.

**Listing 7.36 Testing the custom analyzer**

```
POST index_with_html_strip_filter/_analyze
{
  "text": "<h1>Hello,</h1> <h2>World!</h2>",
  "analyzer": "custom_html_strip_filter_analyzer"
}
```

This code leaves the word with the `<h1>` tags as is and strips the `<h2>` tags. It results in this output: `"<h1>Hello,</h1> World!"`.

### 7.6.2 The mapping character filter

The `mapping` character filter's sole job is to match a key and replace it with a value. In our earlier example of converting Greek letters to English words, the `mapping` filter parsed the symbols and replaced them with words: $\alpha$ as *alpha*, $\beta$ as *beta*, and so on.

Let's test the `mapping` character filter. In the following listing, "UK" is replaced with the "United Kingdom" when parsed with the `mapping` filter.

**Listing 7.37 The `mapping` character filter in action**

```
POST _analyze
{
  "text": "I am from UK",
  "char_filter": [
    {
      "type": "mapping",
      "mappings": [
        "UK => United Kingdom"
      ]
    }
  ]
}
```